

ORDER MANAGEMENT SYSTEM

Sephora Beauty E-commerce Database Management System

1. Database Design

1.1 Orders Table

Attribute	Data Type	Key Type / Constraint
Order_ID	NUMBER	Primary Key
Customer_ID	NUMBER	Foreign Key
Order_Date	DATE	NOT NULL
Total_Amount	NUMBER(10,2)	CHECK (> 0)
Order_Status	VARCHAR2(20)	NOT NULL
Payment_ID	NUMBER	Foreign Key

1.2 Order_Details Table

Attribute	Data Type	Key Type / Constraint
Order_Detail_ID	NUMBER	Primary Key
Order_ID	NUMBER	Foreign Key
Product_ID	NUMBER	Foreign Key
Quantity	NUMBER	CHECK (> 0)
Unit_Price	NUMBER(10,2)	CHECK (> 0)

1.3 Supporting Tables

The Orders and Order_Details tables refer to the following tables through foreign keys, so they are created and filled first.

Customer

Attribute	Data Type	Key Type / Constraint
Customer_ID	NUMBER	Primary Key
First_Name	VARCHAR2(50)	NOT NULL
Last_Name	VARCHAR2(50)	
Email	VARCHAR2(100)	UNIQUE
Phone	VARCHAR2(15)	
City	VARCHAR2(50)	

Product

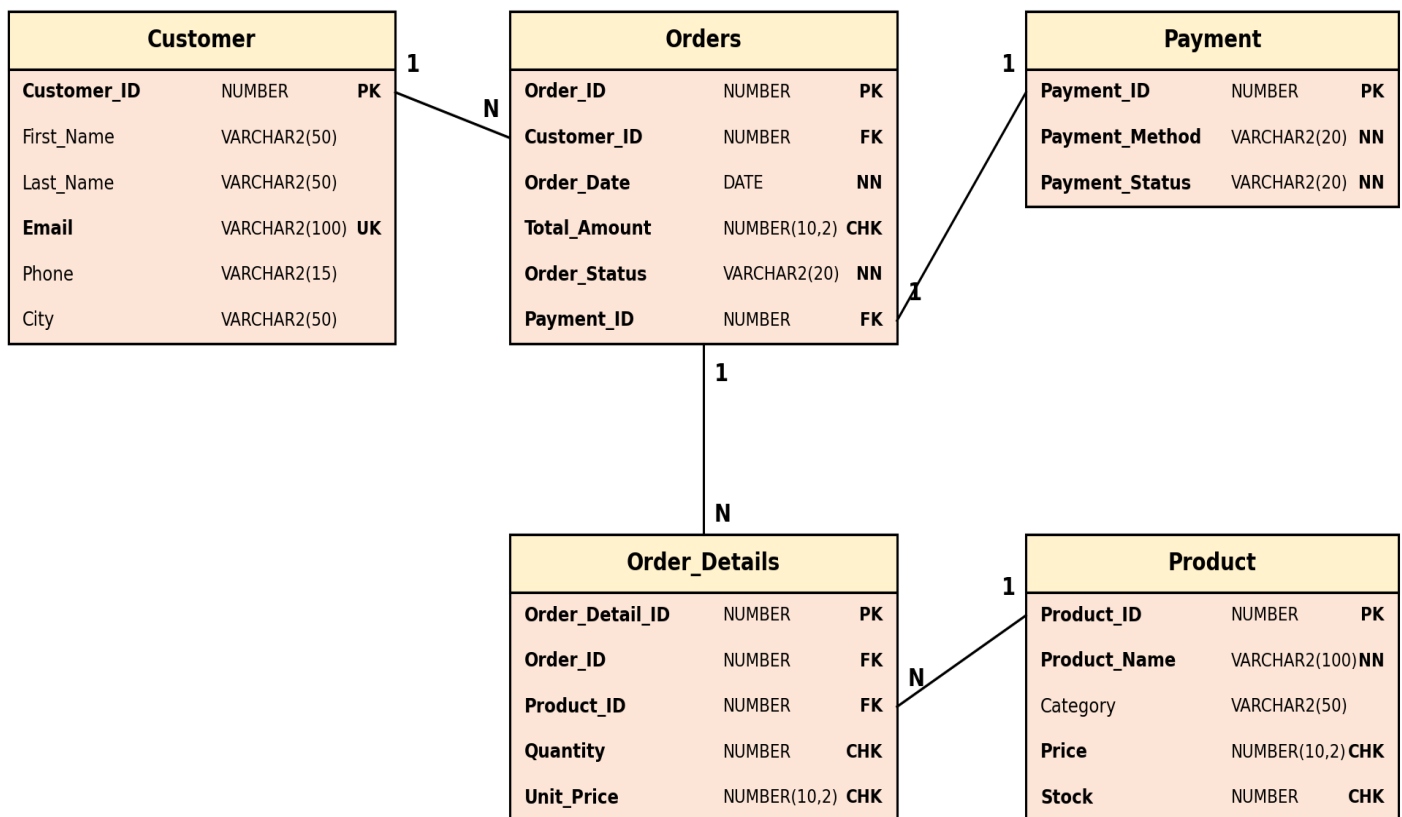
Attribute	Data Type	Key Type / Constraint
Product_ID	NUMBER	Primary Key
Product_Name	VARCHAR2(100)	NOT NULL

Attribute	Data Type	Key Type / Constraint
Category	VARCHAR2(50)	
Price	NUMBER(10,2)	CHECK (> 0)
Stock	NUMBER	CHECK (>= 0)

Payment

Attribute	Data Type	Key Type / Constraint
Payment_ID	NUMBER	Primary Key
Payment_Method	VARCHAR2(20)	NOT NULL
Payment_Status	VARCHAR2(20)	NOT NULL

2. Table Relationships



PK = Primary Key FK = Foreign Key UK = Unique NN = NOT NULL CHK = CHECK constraint

3. Table Creation

- 3.1 Create Supporting Tables One Customer can place many Orders (1 : N) through Customer_ID.
- Each Order is linked to one Payment record (1 : 1) through Payment_ID.
- One Order can contain many Order_Details lines (1 : N) through Order_ID.
- One Product can appear in many Order_Details lines (1 : N) through Product_ID.

```
SQL> CREATE TABLE Customer (
2     Customer_ID NUMBER PRIMARY KEY,
3     First_Name VARCHAR2(50) NOT NULL,
4     Last_Name VARCHAR2(50),
5     Email VARCHAR2(100) UNIQUE,
6     Phone VARCHAR2(15),
7     City VARCHAR2(50)
8 );
```

Table created.

```
SQL> CREATE TABLE Product (
2     Product_ID NUMBER PRIMARY KEY,
3     Product_Name VARCHAR2(100) NOT NULL,
4     Category VARCHAR2(50),
5     Price NUMBER(10,2) CHECK (Price > 0),
6     Stock NUMBER CHECK (Stock >= 0)
7 );
```

Table created.

```
SQL> CREATE TABLE Payment (
2     Payment_ID NUMBER PRIMARY KEY,
3     Payment_Method VARCHAR2(20) NOT NULL,
4     Payment_Status VARCHAR2(20) NOT NULL
5 );
```

Table created.

3.2 Create Orders Table

```
SQL> CREATE TABLE Orders (
2     Order_ID NUMBER PRIMARY KEY,
3     Customer_ID NUMBER,
4     Order_Date DATE NOT NULL,
5     Total_Amount NUMBER(10,2) CHECK (Total_Amount > 0),
6     Order_Status VARCHAR2(20) NOT NULL,
7     Payment_ID NUMBER,
8     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),
9     FOREIGN KEY (Payment_ID) REFERENCES Payment(Payment_ID)
10 );
```

Table created.

3.3 Create Order_Details Table

```
SQL> CREATE TABLE Order_Details (
2     Order_Detail_ID NUMBER PRIMARY KEY,
3     Order_ID NUMBER,
4     Product_ID NUMBER,
5     Quantity NUMBER CHECK (Quantity > 0),
6     Unit_Price NUMBER(10,2) CHECK (Unit_Price > 0),
7     FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID),
8     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)
9 );
```

Table created.

4. Order Data Insertion

4.1 Insert Customer, Product and Payment Data

These rows must exist before any order can be inserted, because of the foreign keys.

```
SQL> INSERT INTO Customer VALUES
2     (101, 'Aarthi', 'Raman', 'aarthi.raman@email.com', '9876500101', 'Chennai');
```

```
1 row created.

SQL> INSERT INTO Customer VALUES
  2      (102, 'Farhan', 'Ali', 'farhan.ali@email.com', '9876500102', 'Bengaluru');

1 row created.

SQL> INSERT INTO Customer VALUES
  2      (103, 'Meena', 'Iyer', 'meena.iyer@email.com', '9876500103', 'Coimbatore');

1 row created.

SQL> INSERT INTO Customer VALUES
  2      (104, 'Karthik', 'Subramanian', 'karthik.s@email.com', '9876500104', 'Madurai');

1 row created.

SQL> INSERT INTO Customer VALUES
  2      (105, 'Divya', 'Lakshmi', 'divya.l@email.com', '9876500105', 'Hyderabad');

1 row created.
```

```
SQL> INSERT INTO Product VALUES
  2      (1, 'Matte Lipstick', 'Makeup', 1500, 50);

1 row created.

SQL> INSERT INTO Product VALUES
  2      (2, 'Vitamin C Serum', 'Skincare', 800, 120);

1 row created.

SQL> INSERT INTO Product VALUES
  2      (3, 'Eau de Parfum 50ml', 'Fragrance', 2500, 40);

1 row created.

SQL> INSERT INTO Product VALUES
  2      (4, 'Hair Care Gift Set', 'Haircare', 3000, 60);

1 row created.

SQL> INSERT INTO Product VALUES
  2      (5, 'Sunscreen SPF 50', 'Skincare', 1200, 80);

1 row created.
```

```
SQL> INSERT INTO Payment VALUES
  2      (1, 'UPI', 'Paid');

1 row created.

SQL> INSERT INTO Payment VALUES
  2      (2, 'Credit Card', 'Paid');

1 row created.

SQL> INSERT INTO Payment VALUES
  2      (3, 'Cash on Delivery', 'Pending');

1 row created.

SQL> INSERT INTO Payment VALUES
  2      (4, 'Net Banking', 'Paid');

1 row created.
```

```
SQL> INSERT INTO Payment VALUES
  2      (5, 'UPI', 'Paid');

1 row created.

SQL> INSERT INTO Payment VALUES
  2      (6, 'Debit Card', 'Paid');

1 row created.
```

4.2 Insert Orders

```
SQL> INSERT INTO Orders VALUES
  2      (1, 101, TO_DATE('2026-09-01','YYYY-MM-DD'), 3000, 'Delivered', 1);

1 row created.

SQL> INSERT INTO Orders VALUES
  2      (2, 102, TO_DATE('2026-09-02','YYYY-MM-DD'), 800, 'Shipped', 2);

1 row created.

SQL> INSERT INTO Orders VALUES
  2      (3, 103, TO_DATE('2026-09-03','YYYY-MM-DD'), 5000, 'Processing', 3);

1 row created.

SQL> INSERT INTO Orders VALUES
  2      (4, 104, TO_DATE('2026-09-04','YYYY-MM-DD'), 3000, 'Delivered', 4);

1 row created.

SQL> INSERT INTO Orders VALUES
  2      (5, 105, TO_DATE('2026-09-05','YYYY-MM-DD'), 3600, 'Shipped', 5);

1 row created.

SQL> INSERT INTO Orders VALUES
  2      (6, 101, TO_DATE('2026-09-06','YYYY-MM-DD'), 800, 'Placed', 6);

1 row created.
```

5. Order_Details Data Insertion

```
SQL> INSERT INTO Order_Details VALUES
  2      (1, 1, 1, 2, 1500);

1 row created.

SQL> INSERT INTO Order_Details VALUES
  2      (2, 2, 2, 1, 800);

1 row created.

SQL> INSERT INTO Order_Details VALUES
  2      (3, 3, 3, 2, 2500);

1 row created.

SQL> INSERT INTO Order_Details VALUES
  2      (4, 4, 4, 1, 3000);

1 row created.
```

```

SQL> INSERT INTO Order_Details VALUES
  2      (5, 5, 5, 3, 1200);

1 row created.

SQL> INSERT INTO Order_Details VALUES
  2      (6, 6, 2, 1, 800);

1 row created.

SQL> COMMIT;

Commit complete.

```

6. Order Modification Operations

6.1 Modify Order Amount

```

SQL> UPDATE Orders
  2  SET Total_Amount = 4500
  3  WHERE Order_ID = 1;

1 row updated.

SQL> UPDATE Orders
  2  SET Total_Amount = 900
  3  WHERE Order_ID = 2;

1 row updated.

SQL> SELECT Order_ID, Total_Amount
  2  FROM Orders
  3  WHERE Order_ID IN (1, 2);

ORDER_ID TOTAL_AMOUNT
-----
1         4500
2         900

```

6.2 Modify Order Data

```

SQL> UPDATE Orders
  2  SET Order_Status = 'Shipped'
  3  WHERE Order_ID = 3;

1 row updated.

SQL> UPDATE Orders
  2  SET Order_Status = 'Processing'
  3  WHERE Order_ID = 6;

1 row updated.

SQL> SELECT Order_ID, Order_Date, Total_Amount, Order_Status
  2  FROM Orders
  3  WHERE Order_ID IN (3, 6);

ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STATUS
-----
3 03-SEP-26      5000 Shipped
6 06-SEP-26       800 Processing

```

7. Order_Details Modification

Order 1 now has 3 lipsticks (3 x 1500 = 4500) and the Vitamin C Serum in order 2 is repriced to 900, so both match the totals updated in section 6.1.

```
SQL> UPDATE Order_Details
  2 SET Quantity = 3
  3 WHERE Order_Detail_ID = 1;

1 row updated.

SQL> UPDATE Order_Details
  2 SET Unit_Price = 900
  3 WHERE Order_Detail_ID = 2;

1 row updated.

SQL> SELECT Order_Detail_ID, Order_ID, Product_ID, Quantity, Unit_Price
  2 FROM Order_Details
  3 WHERE Order_Detail_ID IN (1, 2);
```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	UNIT_PRICE
1	1	1	3	1500
2	2	2	1	900

8. Customer Order History Report

Set the column widths first so every row fits on one line in SQL*Plus:

```
SQL> SET LINESIZE 120
SQL> COLUMN Customer_Name FORMAT A20
SQL> COLUMN Product_Name FORMAT A22
SQL> COLUMN Status FORMAT A10
```

```
SQL> SELECT
  2 c.Customer_ID AS Cust_ID,
  3 c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4 o.Order_ID,
  5 o.Order_Date,
  6 p.Product_Name,
  7 od.Quantity AS Qty,
  8 od.Unit_Price,
  9 o.Total_Amount,
 10 o.Order_Status AS Status
 11 FROM Customer c
 12 JOIN Orders o
 13 ON c.Customer_ID = o.Customer_ID
 14 JOIN Order_Details od
 15 ON o.Order_ID = od.Order_ID
 16 JOIN Product p
 17 ON od.Product_ID = p.Product_ID
 18 ORDER BY c.Customer_ID, o.Order_Date;
```

CUST_ID	CUSTOMER_NAME	ORDER_ID	ORDER_DAT	PRODUCT_NAME	QTY	UNIT_PRICE	TOTAL_AMOUNT	STATUS
101	Aarthi Raman	1	01-SEP-26	Matte Lipstick	3	1500	4500	Delivered
101	Aarthi Raman	6	06-SEP-26	Vitamin C Serum	1	800	800	Processing
102	Farhan Ali	2	02-SEP-26	Vitamin C Serum	1	900	900	Shipped
103	Meena Iyer	3	03-SEP-26	Eau de Parfum 50ml	2	2500	5000	Shipped
104	Karthik Subramanian	4	04-SEP-26	Hair Care Gift Set	1	3000	3000	Delivered
105	Divya Lakshmi	5	05-SEP-26	Sunscreen SPF 50	3	1200	3600	Shipped

9. Customer-wise Order Summary

```
SQL> SELECT
  2 c.Customer_ID,
  3 c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  4 COUNT(o.Order_ID) AS Total_Orders,
  5 SUM(o.Total_Amount) AS Total_Amount
  6 FROM Customer c
  7 JOIN Orders o
  8 ON c.Customer_ID = o.Customer_ID
  9 GROUP BY
```

```
10      c.Customer_ID,  
11      c.First_Name,  
12      c.Last_Name  
13 ORDER BY c.Customer_ID;
```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ORDERS	TOTAL_AMOUNT
101	Aarthi Raman	2	5300
102	Farhan Ali	1	900
103	Meena Iyer	1	5000
104	Karthik Subramanian	1	3000
105	Divya Lakshmi	1	3600